SVI and SVI-JW

Stochastic-volatility-inspired slices, jump-wing handles, and constrained calibration

Vol-Fitter Technical Notes, No. 02

Vol-Fitter

July 11, 2026

Abstract

This note documents the Vol-Fitter's SVI slice model. Raw SVI parametrizes total implied variance as a hyperbola in log-moneyness, $w(k) = a + b\{\rho(k-m) + \sqrt{(k-m)^2 + \sigma_{\mathrm{SVI}}^2}\}$, with five parameters that are geometrically opaque but analytically convenient. The *jump-wing* (SVI-JW) re-coordinatization replaces them with five quantities in trader vocabulary — ATM variance, a time-scaled ATM skew, two normalized wing slopes, and the minimum variance. Product status up front: JW is the analytical and benchmark coordinate system of this note; production fits and stores *raw* SVI and displays model-agnostic ATM handles, and five-handle JW entry, bumping and export are not yet product features. We give the exact conversion in both directions together with its *regular inverse domain*: on $v > 0$, $p, c > 0$, $-p/2 < \psi < c/2$, $\psi \neq 0$, $\widetilde{v} < v$ the displayed formulas define the unique nonsingular inverse — while the image also contains the singular stratum $\psi = 0$ (ATM at the vertex, $\widetilde{v} = v$), attained by infinitely many raw slices, because the handles omit ATM curvature and cannot identify $(m, \sigma_{\mathrm{SVI}})$ there. The production converter carries *no* guards, so the domain is the caller's contract, stated precisely and now test-locked. We restate the two *coded* no-arbitrage fences in JW units — Lee's cap becomes $\max(p, c)\sqrt{v\tau} \leq 2$ and the minimum-variance condition is $\widetilde{v} \geq 0$ — and are precise about what they guarantee: they are finite-weight, adjustable penalties that do *not* certify Durrleman $g \geq 0$ (a classical slice passing both while arbitrageable is test-locked). The unconstrained reparametrization (softplus / tanh / exp) lets a plain Levenberg–Marquardt solver only ever propose *structurally valid hyperbolas*. As in the LQD note, the residual Jacobian is available in closed form; a fresh run recovers a benchmark slice through a production JW→raw→fit→JW round trip to a fraction of a vol bp, and runs 2.50× faster than the finite-difference Jacobian on that microbenchmark.

Contents

1 Product status, conventions, and why two parametrizations

Conventions. Throughout, $k = \log(K/F_T)$ is log-moneyness and $w(k) = \sigma^2_{\text{imp}}(k)\,\tau_T$ total implied variance, where T labels the expiry, t_T is its calendar year fraction and τ_T is the *active variance time* — calendar time dilated by scheduled events (Notes 00, 11), with $\tau_T = t_T$ when the event clock is off. Production passes τ_T (`prepared.tau`) to the SVI calibrator, not the plain calendar maturity; classical formulas below write a bare t, and wherever it plays a time role, read τ_T. The raw SVI width parameter is written σ_{SVI} so it cannot be confused with an implied volatility. A *volatility basis point* (vol bp) is 10^{-4} in volatility. LM is the Levenberg–Marquardt least-squares algorithm and `trf` scipy's trust-region-reflective alternative; $P = 5$ is the SVI parameter count wherever per-parameter costs appear.

Product status. JW is the trader-oriented *analytical* coordinate system used by this note and by the benchmark helper (`jw_to_raw`, whose only repository caller is a benchmark test). Current production calibration fits and stores *raw* SVI as the selected overlay family and displays three model-agnostic numeric handles — ATM vol, skew and curvature, by finite differences of the displayed curve — for every model. There is no backend `raw_to_jw`, no five-handle JW API/UI entry, no JW bumping and no JW export; those are candidate features, not shipped ones. Keep this in mind whenever the prose below says what a desk "quotes" or "reads": it describes the coordinate system, not a wired workflow.

SVI is the workhorse of single-slice equity-index fitting because the hyperbola

$$w(k) = a + b\Big\{\rho(k-m) + \sqrt{(k-m)^2 + \sigma^2_{\text{SVI}}}\Big\} \tag{1}$$

is, in five parameters, simultaneously (i) linear in its wings — $w(k) \sim b(1 \pm \rho)\,|k|$ — so its wing *form* is Lee-compatible whenever the slope cap of section 4 holds, (ii) smooth and convex enough to be butterfly-free over a wide admissible cone, and (iii) cheap to evaluate and differentiate. The cost is that $(a, b, \rho, m, \sigma_{\text{SVI}})$ are geometrically entangled: no single one is "the skew" or "the ATM level". The Vol-Fitter therefore carries SVI in *two* coordinate systems — raw for the optimizer, jump-wing for the analysis — and converts exactly between them on the domain of section 3.3.

Four tiers of "valid". The words matter in this note, so we fix them once:

1. **Structurally valid hyperbola**: $b > 0$, $|\rho| < 1$, $\sigma_{\text{SVI}} > 0$. Guaranteed for *every* optimizer iterate by the reparametrization of section 5 — but note a stays free, so an iterate can still have negative total variance somewhere.

2. **Minimum/Lee clean**: $\min_k w(k) \geq 0$ and $b(1 + |\rho|) \leq 2$. These are the two *coded* penalties — finite-weight, configurable, zero on a clean slice.

3. **Butterfly clean**: $w > 0$ and Durrleman $g(k) \geq 0$ everywhere (with the appropriate boundary behaviour). *Not* implied by tier 2: the counterexample of section 4 passes both screens with $g < 0$.

4. **Calendar clean**: the cross-expiry ordering $w_{T_1}(k) \leq w_{T_2}(k)$ holds; supplied by the model-agnostic hinge of Note 10, again as a finite-weight penalty.

Raw SVI slices can remain arbitrageable without conditions stronger than tier 2 — this is standard in the SVI literature (Gatheral–Jacquier; the full butterfly-domain characterization is Martini–Mingone).

Invariant

1. The optimizer can only propose structurally valid hyperbolas (tier 1): $b \geq 0$, $|\rho| < 1$, $\sigma_{\text{SVI}} > 0$ hold by reparametrization, not by rejection.
2. The two coded no-arbitrage fences (tier 2) — non-negative minimum variance and the Lee wing cap — are soft hinges that are *exactly zero* on any tier-2-clean slice (byte-identical fits). They bound, but do not certify, tier-3 butterfly cleanliness.
3. The raw↔JW conversion is exact on its *regular* domain (proposition 1); the converter itself carries no guards, so the domain is the caller's contract (test-locked).
4. SVI is a display overlay like any other: the band, calendar, var-swap, prior and extrapolation-fence blocks share the same product targets as the other models, expressed through *model-specific* residuals (SVI works in IV units; LQD in vega-normalized price; the var-swap routes differ). Each is absent-is-byte-identical.

Heuristic

Read (1) as a hyperbola: b is the overall steepness (the asymptotic opening angle), $\rho \in (-1, 1)$ tilts it (left wing steeper than right for the equity put skew $\rho < 0$), m shifts its vertex in k, σ_{SVI} rounds off the vertex (larger σ_{SVI} = flatter bottom), and a raises the whole curve. The wings are straight lines of slope $b(1-\rho)$ on the left and $b(1+\rho)$ on the right — which is exactly what Lee's bound constrains.

2 Raw SVI geometry

The derivatives used everywhere below are

$$w'(k) = b\Big(\rho + \frac{k-m}{r}\Big), \qquad w''(k) = \frac{b\,\sigma_{\text{SVI}}^2}{r^3}, \qquad r = \sqrt{(k-m)^2 + \sigma_{\text{SVI}}^2}. \tag{2}$$

Note $w'' > 0$ always (the slice is one globally convex curve in k, with a single minimum — the geometric fact behind the model's rigidity on event shapes, section 7), and the wing limits are

$$\lim_{k\to-\infty} \frac{w(k)}{|k|} = b(1-\rho), \qquad \lim_{k\to+\infty} \frac{w(k)}{k} = b(1+\rho). \tag{3}$$

The vertex (minimum total variance) sits at $k^\star = m - \sigma_{\text{SVI}}\rho/\sqrt{1-\rho^2}$ with value $w(k^\star) = a + b\sigma_{\text{SVI}}\sqrt{1-\rho^2}$; this minimum is the quantity the jump-wing parametrization calls $\tilde{v}\,t$.

3 The jump-wing handles

The SVI-JW parametrization $(v, \psi, p, c, \widetilde{v})$ at expiry t is defined by ATM and wing functionals of the slice:

$$\begin{aligned} v &= \frac{w(0)}{t} && \text{ATM variance,} \\ \psi &= \frac{b}{2\sqrt{w_0}}\Big(\rho - \frac{m}{\sqrt{m^2+\sigma_{\mathrm{SVI}}^2}}\Big) && \text{ATM skew (time-scaled),} \\ p &= \frac{b(1-\rho)}{\sqrt{w_0}}, \quad c = \frac{b(1+\rho)}{\sqrt{w_0}} && \text{put/call wing slopes (normalized),} \\ \widetilde{v} &= \frac{a + b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2}}{t} && \text{minimum variance,} \end{aligned} \tag{4}$$

with $w_0 = w(0) = vt$.

3.1 What the handles are, in displayed units

None of the five handles is a directly plotted quantity, and misreading their units is the most common JW error. At the production variance time τ:

$$\sigma_{\mathrm{ATM}} = \sqrt{v}, \qquad \partial_k \sigma_{\mathrm{imp}}(0) = \frac{\psi}{\sqrt{\tau}}, \qquad \beta_L = p\sqrt{v\tau}, \qquad \beta_R = c\sqrt{v\tau}, \qquad \sigma_{\min} = \sqrt{\widetilde{v}}, \tag{5}$$

where $\beta_{L,R}$ are the asymptotic *total-variance* slopes $b(1 \mp \rho)$ of (3). In words: v and $\widetilde{v}$ are *variances* (the IV chart shows their square roots); ψ is a *time-scaled skew handle*, not the displayed IV tangent — the actual ATM tangent slope is $\psi/\sqrt{\tau}$; and p, c are *normalized total-variance wing slopes*, not slopes of the IV curve. Figure 1 annotates a fitted slice in exactly these displayed units, with the wing slopes on a total-variance inset where they actually live.

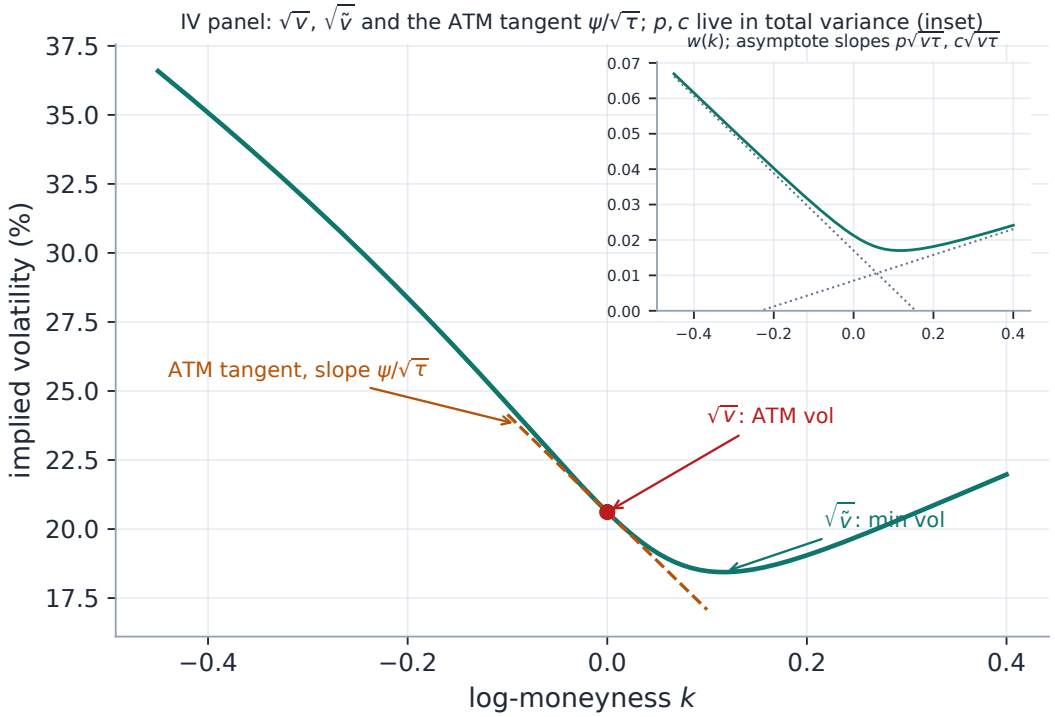


Figure 1: The five SVI-JW handles in *displayed* units. Main panel (IV): the ATM level $\sqrt{v}$, the minimum $\sqrt{\tilde{v}}$, and the ATM tangent of slope $\psi/\sqrt{\tau}$. Inset (total variance $w(k)$): the two asymptotes whose slopes are $\beta_L = p\sqrt{v\tau}$ and $\beta_R = c\sqrt{v\tau}$ — the wing handles are total-variance objects, not IV slopes.

3.2 The conversion, both ways

The forward direction, jump-wing *to* raw, is what the shipped converter `jw_to_raw` computes (today its only repository caller is the benchmark test; see the product-status paragraph of section 1). Set $w_0 = vt$, $\sqrt{w_0} = \sqrt{vt}$, and

$$b = \frac{\sqrt{w_0}}{2}(p + c), \qquad \rho = \frac{c - p}{c + p}, \qquad \chi := \frac{m}{\sqrt{m^2 + \sigma_{\mathrm{SVI}}^2}} = \rho - \frac{4\psi}{p + c}. \tag{6}$$

From χ one recovers $m = \chi\sigma_{\mathrm{SVI}}/\sqrt{1-\chi^2}$ and $\sqrt{m^2 + \sigma_{\mathrm{SVI}}^2} = \sigma_{\mathrm{SVI}}/\sqrt{1-\chi^2}$; subtracting the minimum-variance identity from the ATM identity gives the closed form

$$\sigma_{\mathrm{SVI}} = \frac{w_0 - \tilde{v}\,t}{b\big((1-\rho\chi)/\sqrt{1-\chi^2} - \sqrt{1-\rho^2}\big)}, \qquad a = \tilde{v}\,t - b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2}. \tag{7}$$

The reverse map *from* raw *to* jump-wing is just the definitions (4) evaluated on the fitted raw slice; the backend ships no `raw_to_jw` — that direction lives in the figure generator and in these equations. Section 6 runs the production round trip numerically. The forward conversion (6)–(7) is direct:

Listing 1: SVI-JW handles → raw SVI (distilled from `models/svi_jw/svi.py`; agrees with it to 10^{-12}).

```python
def jw_to_raw(t, v, psi, p, c, vt):
    # (v, psi, p, c, vtilde) -> (a, b, rho, m, sigma)
    w0 = v*t; sq = np.sqrt(w0)
    b = 0.5*sq*(p + c)
    rho = (c - p)/(c + p)
    chi = rho - 4*psi/(p + c)       # m / sqrt(m^2 + sigma^2)
    r1x = np.sqrt(1 - chi*chi)
    r1r = np.sqrt(1 - rho*rho)
    sigma = (w0 - vt*t) / (b*((1 - rho*chi)/r1x - r1r))
    m = chi*sigma/r1x
    a = vt*t - b*sigma*r1r
    return a, b, rho, m, sigma
```

3.3 The regular inverse domain, exactly

Not every quintuple $(v, \psi, p, c, \widetilde{v})$ is a smile, and — a finer point — *regular invertibility* is not the same as *representability*. The formulas above divide by $p + c$, by $\sqrt{1-\chi^2}$ and by the bracket in (7), and the production converter carries *no* guards — so the exact domain is worth stating and proving.

Proposition 1 (Regular inverse domain)**.** *Fix $t > 0$. On*

$$v > 0, \qquad p > 0,\ c > 0, \qquad -\frac{p}{2} < \psi < \frac{c}{2}, \qquad \psi \neq 0, \qquad \widetilde{v} < v,$$

the map (6)–(7) defines the unique, nonsingular inverse of the JW definitions (4): it produces a structurally valid raw slice ($b > 0$, $|\rho| < 1$, $\sigma_{\mathrm{SVI}} > 0$, m finite), and outside this set the displayed formulas produce no such slice. In words: the wings must be genuinely upward, the quoted ATM skew must lie strictly between half the wing slopes*, and the ATM variance must sit strictly above the minimum.*

Two complements make the full picture. First, *representability* is larger than regular invertibility: the image of the JW map also contains the singular stratum $\psi = 0$, $\widetilde{v} = v$ (remark 1 below), realized by *infinitely many* raw slices — so "the formulas fail" does not mean "no slice has these handles". Second, a financially usable smile additionally requires $\widetilde{v} \geq 0$ (tier 2 of section 1); that is an economic condition, separate from the algebraic invertibility above.

Proof. $p, c > 0$ gives $b = \frac{\sqrt{w_0}}{2}(p + c) > 0$ and $\rho = (c - p)/(c + p) \in (-1, 1)$. For $\chi = \rho - 4\psi/(p + c)$, using $(\rho - 1)(p + c) = -2p$ and $(\rho + 1)(p + c) = 2c$,

$$|\chi| < 1 \iff \frac{(\rho - 1)(p + c)}{4} < \psi < \frac{(\rho + 1)(p + c)}{4} \iff -\frac{p}{2} < \psi < \frac{c}{2},$$

which keeps $m = \chi\sigma_{\mathrm{SVI}}/\sqrt{1-\chi^2}$ finite. For σ_{SVI}: by Cauchy–Schwarz applied to the unit vectors $(\rho, \sqrt{1-\rho^2})$ and $(\chi, \sqrt{1-\chi^2})$,

$$\rho\chi + \sqrt{1-\rho^2}\sqrt{1-\chi^2} \leq 1, \quad \text{i.e.} \quad \frac{1 - \rho\chi}{\sqrt{1-\chi^2}} \geq \sqrt{1-\rho^2},$$

with equality iff $\chi = \rho$, i.e. iff $\psi = 0$. So the bracket in (7) is strictly positive exactly when $\psi \neq 0$, and $\sigma_{\mathrm{SVI}} > 0$ then requires the numerator $w_0 - \widetilde{v}t > 0$, i.e. $\widetilde{v} < v$. Conversely each failure breaks a

requirement. If $p+c<0$ then $b<0$; if $p+c=0$ the very first formula is undefined; and if $p+c>0$ with one of $p,c\le 0$ then $|\rho|=|c-p|/(c+p)\ge 1$ — so $p,c>0$ is forced. ψ outside the band sends $|\chi|\ge 1$ (m infinite); $\psi=0$ zeroes the bracket; $\widetilde{v}\ge v$ makes $\sigma_{\mathrm{SVI}}\le 0$. □

Remark 1 (The $\psi=0$ stratum is a genuine coordinate singularity)**.** On a true SVI slice, $\psi=0$ means $\chi=\rho$, which happens exactly when the ATM point *is* the vertex — and then $\widetilde{v}=v$ automatically. (Call it the *ATM-at-the-vertex* stratum: $k^\star=0$ implies no symmetry unless $\rho=0$.) On that stratum the five handles do not identify the slice: for any $\sigma_{\mathrm{SVI}}>0$, the raw slice with $m=\rho\sigma_{\mathrm{SVI}}/\sqrt{1-\rho^2}$ and $a=\widetilde{v}t-b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2}$ reproduces the *same* $(v,0,p,c,v)$ — the handles omit ATM curvature, and this is where it shows (test-locked: `test_svi_domain.py::test_psi_zero_stratum_not_identified`). The jump-wing chart is a fine coordinate system *away* from the ATM-at-the-vertex stratum, and ill-conditioned near it: as $\psi\to 0$, the conversion computes σ_{SVI} as a ratio of two vanishing quantities, with catastrophic cancellation before the limit.

> **Caution**
>
> The production `jw_to_raw` implements the formulas verbatim with *zero* domain checks, and the failure modes differ by case — none is uniformly "silent": a scalar $p+c=0$ raises `ZeroDivisionError`; ψ outside $(-p/2,c/2)$ takes the square root of a negative and returns NaNs (with a runtime warning); $\widetilde{v}\ge v$ returns $\sigma_{\mathrm{SVI}}\le 0$, not a slice; and a near-zero ψ silently amplifies rounding. All of this is deliberate — the converter's only caller feeds it benchmark handles comfortably inside proposition 1 — but it makes the domain a *contract*, not a runtime guarantee. The contract and each failure mode are now test-locked (`test_svi_domain.py`); anyone wiring JW handles to user input must still validate against proposition 1 first.

4 Static no-arbitrage

4.1 Butterfly: Durrleman's function

For a positive, sufficiently smooth total-variance slice w with the appropriate wing behaviour (the large-$|k|$ limits that make the density integrate to one — satisfied by any structurally valid SVI slice with $w>0$), the slice is free of butterfly arbitrage iff the risk-neutral density it implies is non-negative, which Durrleman's condition expresses directly on total variance:

$$g(k)=\Big(1-\frac{k\,w'(k)}{2w(k)}\Big)^2-\frac{w'(k)^2}{4}\Big(\frac{1}{w(k)}+\frac{1}{4}\Big)+\frac{w''(k)}{2}\ \ge\ 0\quad\forall k. \tag{8}$$

For raw SVI, g is a smooth function of (1)–(2) with no closed-form sign, and where each consumer evaluates it matters: the backtest's arbitrage metric evaluates the closed-form (w,w',w'') triple (Note 09); the application's extrapolated-region diagnostic finite-differences the smooth *model* w (never reconstructed option prices); this note's figures use the closed form. The *default* SVI fit itself carries no g hinge and samples no g rows — the Gatheral–Jacquier parameter-space conditions bound the wing steepness $b(1+|\rho|)$ and the minimum total variance, and the calibrator's core fences are the two that usually bite.

That "usually" is load-bearing, and there is a classical counterexample: the Gatheral–Jacquier slice $(a,b,\rho,m,\sigma_{\mathrm{SVI}})=(-0.0410,\ 0.1331,\ 0.3060,\ 0.3586,\ 0.4153)$ has positive minimum variance (0.0116) and a Lee slope far under the cap (0.174), yet $g(k)$ dips to -0.033 near $k\approx 0.88$: both

coded screens pass while genuine butterfly arbitrage remains (test-locked in `test_svi_domain.py`; full anchor in the traceability table).

How g is *measured* matters as much as what it says. An earlier backtest metric evaluated g by finite differences of reconstructed option prices, and its numerical noise flagged 28.3% of provably-clean LQD slices as arbitrageable; recomputed analytically from each model's own (w, w', w''), the LQD rate reads 0.0%, and SVI's flagged rate fell from 20.8% to 9.2% — the survivors being genuine: the soft hinges fence the conditions that bite, they do not certify $g \geq 0$ pointwise. The production rule that came out of that audit is that the diagnostic is always computed on model derivatives, never by differencing prices (Note 09 has the cross-model treatment).

4.2 The four production layers

What production actually runs is a stack of four distinct layers, in increasing distance from the core fit:

1. **Core fences** (always in the objective): the min-variance and Lee penalty rows of section 4.3 — finite-weight, configurable, zero on a clean slice.

2. **Extrapolated-region enforcement** (`extrapEnforce`, default *off*; Notes 09–10): opt-in tapered residual rows on the extrapolated strike region — a Durrleman-g hinge on the slice's own envelope, a tapered calendar hinge against the previous displayed slice, and the wing-slope-order hinge. These rows are finite-differenced while the core SVI rows stay analytic, so the fit runs a *hybrid* Jacobian (`calib/extrap.py`; `models/svi_jw/calibrate.py`).

3. **Advisory diagnostics** (always on): the Quality tab measures and reports remaining extrapolated-region $g < 0$ per fit — measurement, not enforcement.

4. **Publish-time wing projection** (export-only; Note 09 Phase 3): a discrete projection of the exported wing samples that raises prices only, leaving the fitted core pinned.

None of these layers turns a fitted SVI slice into a certified butterfly-free object; together they bound, measure, and clean at the boundary where it matters.

Proposition 2 (The enforced conditions, in JW units)**.** *Under the conversion of section 3, the two coded penalties read directly on the trader handles: the minimum-variance condition* $a + b\sigma_{\mathrm{SVI}}\sqrt{1-\rho^2} \geq 0$ *is exactly* $\widetilde{v} \geq 0$*, and Lee's cap* $b(1+|\rho|) \leq \beta_{\max}$ *is exactly*

$$\max(p, c)\,\sqrt{vt} \;\leq\; \beta_{\max}\ (= 2).$$

Proof. The first is the definition of $\widetilde{v}$ in (4) times $t > 0$. For the second, $b(1 \mp \rho) = p\sqrt{w_0}$ and $c\sqrt{w_0}$ respectively by (4), so $b(1+|\rho|) = \max\big(b(1-\rho), b(1+\rho)\big) = \max(p,c)\sqrt{w_0}$ with $w_0 = vt$. □

So a desk can sanity-check a handle quote before any conversion: wings non-negative at the vertex ($\widetilde{v} \geq 0$), skew inside $(-p/2,\ c/2)$ (proposition 1), and $\max(p,c)\sqrt{vt} \leq 2$ (Lee).

4.3 The two soft penalties

The optimizer adds, to the data residual, two penalty rows that are *exactly zero* on a tier-2-clean slice and grow linearly past the boundary:

$$\rho_{\text{pen}}(\theta) = W \begin{pmatrix} \max\left(-(a + b\sigma_{\text{SVI}}\sqrt{1-\rho^2}),\, 0\right) \\ \max\left(b(1+|\rho|) - \beta_{\max},\, 0\right) \end{pmatrix}, \tag{9}$$

with residual multiplier $W = 1000$ and Lee cap $\beta_{\max} = 2.0$. The first prevents *negative total variance at the minimum*; the second enforces Lee's bound that the asymptotic total-variance slope cannot exceed 2. Because both vanish on a clean fit, a tier-2-clean smile (such as the benchmark of section 6) is byte-identical with or without them — they are a feasibility fence, not a regularizer.

> **Caution**
>
> Read the semantics of W carefully: it multiplies the *residual* rows in (9), so the least-squares objective receives an effective W^2 coefficient on the squared violation — and the two rows do not even share a natural financial unit (one is a total variance, the other a dimensionless slope), so W is a residual multiplier, not a "weight in vol^2 units". It is deliberately large enough to dominate a violated constraint yet invisible on a clean slice. It is *not* a tuning dial for fit quality: raising it cannot improve a feasible fit, and lowering it only lets the optimizer wander into the arbitrageable cone. Both fences are also *adjustable*, not unconditional: a user can set `sviPenaltyWeight` $= 0$ (fences off) or pick a `leeSlopeMax` above 2, so they are configuration, not guarantees. Note 09 develops the Lee bound across all the models; here it is one of these two hinges.

5 Calibration

5.1 Unconstrained reparametrization

Rather than run a bound-constrained optimizer, the Vol-Fitter reparametrizes so that *every* point of $\mathbb{R}^5$ maps to a *structurally valid raw hyperbola* (tier 1 of section 1):

$$a \text{ free}, \quad b = \text{softplus}(\theta_b) \geq 0, \quad \rho = \tanh(\theta_\rho) \in (-1, 1), \quad m \text{ free}, \quad \sigma_{\text{SVI}} = e^{\theta_\sigma} > 0. \tag{10}$$

This guarantees $b \geq 0$, $|\rho| < 1$, $\sigma_{\text{SVI}} > 0$ for free, so the unconstrained Levenberg–Marquardt solver never wastes effort on the box and never proposes a nonsensical hyperbola. It guarantees no more than that: a stays free, so an optimizer iterate can still carry negative total variance somewhere (the vol conversion floors w at 10^{-12} so the residual stays evaluable), and arbitrage cleanliness is the business of the fences of section 4, which remain soft.

5.2 Objective, weights and the data-driven start

The data term is a plain implied-volatility residual, $r_i = \sqrt{\omega_i}\,(\sigma^{\text{SVI}}(k_i) - \sigma_i)$. The surfaced weighting choices are `weightScheme` $\in$ {`equal`, `tv_density`} (Note 07) — the same two schemes every model gets; the calibrator's low-level API also accepts arbitrary ω_i (e.g. vega^2, used by tests and research scripts), but that is a code capability, not a surfaced setting. The initializer is geometric and reads the smile directly: the argmin of w seeds (a, m), and the two finite wing slopes seed (b, ρ) via (3),

then the reparametrization (10) is inverted to get θ_0. On a liquid smile a single LM pass converges from this start.

5.3 The analytic Jacobian, derived

The analytic Jacobian is worth deriving rather than asserting; it is three chain-rule layers stacked (`models/svi_jw/jacobian.py`). With $x = k - m$ and $r = \sqrt{x^2 + \sigma_{\text{SVI}}^2}$, the raw-parameter partials of the hyperbola are

$$\frac{\partial w}{\partial a} = 1, \quad \frac{\partial w}{\partial b} = \rho x + r, \quad \frac{\partial w}{\partial \rho} = b\,x, \quad \frac{\partial w}{\partial m} = -b\Big(\rho + \frac{x}{r}\Big), \quad \frac{\partial w}{\partial \sigma_{\text{SVI}}} = \frac{b\,\sigma_{\text{SVI}}}{r}. \tag{11}$$

The data rows live in IV units, so the second layer is the Black chain rule for $\sigma_{\text{imp}} = \sqrt{w/\tau}$:

$$\frac{\partial \sigma_{\text{imp}}}{\partial w} = \frac{1}{2\sqrt{w\,\tau}} = \frac{1}{2\,\tau\,\sigma_{\text{imp}}}, \tag{12}$$

with the gradient zeroed wherever the $w \le 10^{-12}$ floor is active (the clamp flattens it). The third layer is the reparametrization (10):

$$\frac{\mathrm{d}b}{\mathrm{d}\theta_b} = \text{sigmoid}(\theta_b) = 1 - e^{-b}, \qquad \frac{\mathrm{d}\rho}{\mathrm{d}\theta_\rho} = 1 - \rho^2, \qquad \frac{\mathrm{d}\sigma_{\text{SVI}}}{\mathrm{d}\theta_\sigma} = \sigma_{\text{SVI}}. \tag{13}$$

The hinge rows use the subgradient convention "active linear part, else zero": a penalty/band/calendar $\max(\cdot, 0)$ row contributes its full linear derivative where the violation is strictly positive and a zero row otherwise (at the kink itself the row is treated as inactive). Concretely, the min-variance row differentiates $-(a + b\sigma_{\text{SVI}}\sqrt{1-\rho^2})$ — e.g. its θ_ρ entry is $+Wb\sigma_{\text{SVI}}\rho\sqrt{1-\rho^2}$ after (13) — and the Lee row differentiates $b(1 + |\rho|)$ with the $\text{sign}(\rho)$ subgradient for $|\rho|$. The calibrator uses this Jacobian whenever no var-swap or prior block is present; those blocks are not differentiated and switch the fit to scipy's finite differences, exactly as in LQD. With `extrapEnforce` on, the fit is *hybrid*: the rows above stay analytic and only the small extrapolation block is finite-differenced.

5.4 The optional blocks

The optional blocks share the same product targets as the rest of the series, expressed through model-specific residuals:

- **Band fit** (Note 07): the mid residual becomes a vol-space band hinge with a small mid anchor.
- **Calendar hinge** (Note 10): the *model-agnostic* constraint $\sqrt{W_{\text{cal}}}\,\max(\text{floor} - w(k), 0)$ penalizes calendar crossings against the nearer expiry — the same product target LQD expresses via its quantile $A(z)$, here written directly on $w(k)$. Being a finite-weight hinge, it drives crossings down rather than provably to zero; the locking tests require a substantial reduction of the violation, not exact elimination.
- **Extrapolation fences** (`extrapEnforce`, Notes 09–10): the tapered extrapolated-region rows of section 4.2, default off, hybrid-Jacobian when on.
- **Var-swap target** (Note 08) and **prior persistence** (Note 13): the same targets as LQD through SVI-native residuals (the var-swap routes differ per model: LQD uses its closed-form

quantile moment, SVI the strike replication) — so the SVI *display overlay* receives the same prior treatment as the primary model, closing a long-standing asymmetry in which SVI overlays got no prior at all.

Performance

The solver is Levenberg–Marquardt (`method="lm"`), not trust-region reflective. On noisy real chains LM crosses the penalty kinks in far fewer iterations than `trf` at matched tolerance, so it is the faster same-accuracy choice; the analytic Jacobian is then a drop-in that swaps scipy's $1 + P$ finite-difference evaluations ($P = 5$) for one closed-form call. It shipped after the offline backtest (project `backtest/`) identified those finite-difference evaluations as SVI's dominant fit cost. Two measurements, labeled for what they are: (i) a *historical real-node* result — 26.3 → 10.2 ms per fit ($\approx 2.6\times$) on spike-regime backtest nodes, June-2026 harness configuration, same optimizer and tolerances, same solution within fit precision; and (ii) this note's *private residual/Jacobian microbenchmark* — 2.50× on the 25-quote synthetic benchmark (section 6), single-threaded on the development machine, core rows only, costs agreeing to $2.7e-33$. See appendix B.

6 Worked example: noise-free raw-SVI recovery and JW readout

The cleanest test of an SVI-JW implementation is a round trip through the shipped converter, and since this pass the figure generator runs exactly that: the target raw slice is built by the *production* `jw_to_raw` from the jump-wing handles $(v, \psi, p, c, \tilde{v}) = (0.0425, -0.25, 0.75, 0.25, 0.034)$ at $t = 0.5$, giving $(a, b, \rho, m, \sigma_{\mathrm{SVI}}) \approx (0.010625,\ 0.07289,\ -0.5,\ 0.05831,\ 0.10100)$. Generate noise-free quotes from it, refit raw SVI from the geometric start, and read the handles back through the JW definitions — a JW→raw→fit→JW loop in which the forward leg is production code.

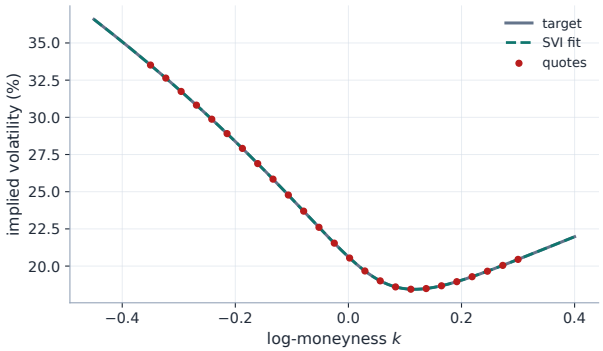
The fit recovers the raw parameters (table 1) with a maximum implied-volatility error of 0.00 volatility basis points in 21 residual evaluations, the wing slopes come out $b(1-\rho) = 0.1093$ (put) and $b(1+\rho) = 0.0364$ (call), and — the point of the exercise — the recovered jump-wing handles (table 2) reproduce the original $(0.0425, -0.25, 0.75, 0.25, 0.034)$ to display precision. The conversion (4)–(7) is therefore exact on this benchmark, not approximate. Figure 2(a) shows the fit and fig. 2(b) the Durrleman function staying non-negative on the diagnostic grid $k \in [-0.45, 0.40]$.

Table 1: Recovered raw SVI.

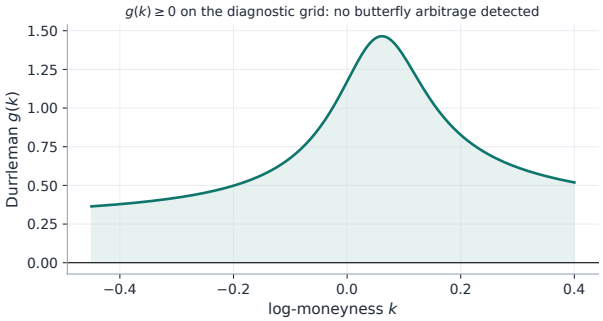
Parameter	Value
a	+0.010625
b	+0.072887
ρ	-0.500000
m	+0.058310
σ	+0.100995

Table 2: Recovered SVI-JW handles.

Handle	Value
v	+0.042500
ψ	-0.250000
p	+0.750000
c	+0.250000
$\tilde{v}$	+0.034000



(a) target, fit, quotes



(b) Durrleman $g(k) \geq 0$

Figure 2: Left: the benchmark smile and its SVI fit (max error 0.00 vol bps). Right: the butterfly diagnostic $g(k)$, non-negative on the plotted diagnostic grid.

Heuristic

A recovery error of 0.00 vol bps is expected because the target *is* a noise-free SVI slice — the exercise validates the optimizer and the conversion, not the expressiveness of SVI, and says nothing about market fit. On real NBBO history the published spike-regime replay measured SVI-JW at 24.3 vol bp RMS in-sample and 26.8 out-of-sample across 1,576 nodes (`backtest/results`); SVI is also stiffer than LQD or Local-Vol on event or double-hump shapes (one globally convex hyperbola, one vertex), which is precisely why the app offers the other models.

7 What is original here, and limitations

SVI and the jump-wing handles are due to Gatheral; the arbitrage-free analysis is Gatheral–Jacquier and Durrleman. The implementation choices worth noting are the *structural-validity-by-reparametrization* (10) (the solver cannot leave tier 1 except through the two soft economic fences), the *exact raw↔JW conversion with a proved regular domain* (the analytical spine of this note and the benchmark; not yet a wired trader workflow — see section 1), and the *analytic LM Jacobian* that makes SVI the speed peer of the other models.

Limitations, stated plainly. Raw SVI total variance is *one globally convex hyperbola with a single minimum*, so it cannot reproduce a WW-shaped total-variance smile (Note 03) and can be too rigid for event-shaped targets (Note 01's double-hat); its single global curve can also over-smooth a sharp short-dated skew. It is butterfly-clean only over a cone the coded fences do not fully certify (section 4), and carries no calendar coupling on its own — that is supplied, softly, by the model-agnostic hinge of Note 10. On sparse chains two further cautions apply: the initializer's "wing slopes" are finite-span proxies read off the quoted range, not asymptotic observations, and a narrow or one-sided quote set can leave the raw parameters poorly identified — distinct $(a, b, \rho, m, \sigma_{\text{SVI}})$ pricing the quoted range indistinguishably — even when the fitted *curve* is stable. Compare curves and handles across fits, not raw coefficients.

8 Traceability

A word on what these tests do and do not prove, so the anchors are not over-read. The benchmark-fit tests exercise an *already tier-2-clean* target, so the "enforcement" rows lock that clean fits are untouched and that the penalty *rows and Jacobians* are correct when activated — they do not demonstrate that an adversarial fit ends feasible. The recovery test whose name says "machine precision" asserts parameter agreement at $\sim 10^{-4}$ relative and curve agreement at $\sim 10^{-6}$ — the freshly generated example itself does reach machine precision, but the *lock* is looser than its name. The conversion-domain contract, previously untested, is now locked by `test_svi_domain.py`, including the classical screens-pass-but-$g < 0$ counterexample.

Table 3: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor *Test anchors*
JW→raw conversion exact on the benchmark	section 3	`models/svi_jw/svi.py::jw_to_raw` `test_lqd_pricing.py::test_jw_conversion_matches_note`

Regular inverse domain; $\psi = 0$ non-identification; invalid-input behaviour	proposition 1, remark 1	`models/svi_jw/svi.py`
		`test_svi_domain.py::test_regular_domain_round_trips`
		`::test_psi_zero_stratum_not_identified`
		`::test_invalid_inputs_fail_loudly_not_plausibly`

Core screens do not certify butterfly freedom (adversarial slice)	section 4	`models/svi_jw/calibrate.py::_penalties` `test_svi_domain.py::test_core_screens_do_not_certify_butterfly_freedom`

Noise-free benchmark recovery (params $\sim 10^{-4}$ rel, curve $\sim 10^{-6}$)	section 6	`models/svi_jw/calibrate.py`
		`test_svi_calibrate.py::test_recovers_benchmark_parameters`
		`::test_curve_reproduced_to_machine_precision`
		(name predates the looser lock)

Clean fits untouched by the fences (benchmark is tier-2-clean)	proposition 2	`models/svi_jw/calibrate.py::_penalties`
		`test_svi_calibrate.py::test_respects_lee_wing_bound`
		`::test_min_variance_non_negative`

Analytic Jacobian matches FD, penalty rows active or not (row correctness, not end-state feasibility)	section 5.3	`models/svi_jw/jacobian.py`
		`test_svi_jacobian.py::test_mid_fit_admissible`
		`::test_lee_penalty_active`
		`::test_min_variance_penalty_active`

Calendar hinge rows correct; off is byte-identical	section 5	`models/svi_jw/calibrate.py`
		`test_svi_jacobian.py::test_calendar_floor_active`
		`test_overlay_calendar.py::test_svi_no_floor_is_byte_identical`

Claim	Object	Code anchor *Test anchors*
Arbitrary per-quote weights accepted (low-level API; not a surfaced setting)	section 5	`models/svi_jw/calibrate.py::_vega_weights` `test_svi_calibrate.py::test_recovers_under_vega_weights`
Prior blocks reach the SVI overlay	section 5	`models/svi_jw/calibrate.py` `test_prior_parametric.py::test_operator_prior_pulls_all_models_toward_prior_skew`

A Hyperparameter atlas

Table 4: SVI hyperparameters.

Knob	Default	Role
Surfaced (FitSettings / OptionsSettings)		
`sviPenaltyWeight` W	1000	Residual multiplier of the two soft no-arbitrage rows (9) (effective W^2 in the squared cost; 0 = fences off).
`leeSlopeMax` $\beta_{\max}$	2.0	Lee wing-slope cap; the asymptotic total-variance slope $b(1+\lvert\rho\rvert)$ may not exceed it (values > 2 are accepted — a fence, not a guarantee).
`midAnchorWeight`	0.05	Mid anchor weight in the band-fit objective (Note 07).
`weightScheme`	`equal`	Per-quote weights, `equal` / `tv_density` (Note 07). Arbitrary weights (e.g. vega^2) exist only at the low-level API, not as a setting.
`calendarWeight`	10^6	Calendar hinge penalty (Note 10; lives in `OptionsSettings`, not `FitSettings`).
`extrapEnforce`	off	Tapered extrapolated-region fences (section 4.2; Notes 09–10). Calibration-affecting; hybrid Jacobian when on.
Internal (reparametrization / solver)		
$b = \mathrm{softplus}(\theta_b)$	—	Enforces $b \geq 0$.
$\rho = \tanh(\theta_\rho)$	—	Enforces $\lvert\rho\rvert < 1$.
$\sigma_{\mathrm{SVI}} = e^{\theta_\sigma}$	—	Enforces $\sigma_{\mathrm{SVI}} > 0$.
LM tolerances	10^{-15}	`xtol/ftol/gtol`; LM converges in few iterations so tight tolerances are cheap.
floor 10^{-12}	—	Total-variance floor inside $\sqrt{w/t}$ to avoid $\sqrt{\text{neg}}$ on structurally-valid-but-negative-variance trial slices.

B Performance notes

1. **Levenberg–Marquardt over trust-region.** On real noisy chains LM crosses the penalty kinks in fewer iterations than `trf` at matched tolerance; `trf` at 10^{-10} was measured *slower* on real nodes.

2. **Analytic Jacobian.** Closed-form Jacobian of the core residual stack (data + penalties + calendar) through the reparametrization. Two measurements, with their scopes: the note's *residual/Jacobian microbenchmark* (25 synthetic quotes, core rows only, single-threaded development machine, fastest of three) measured 1.1 ms vs 2.8 ms (2.50×) with the costs agreeing to $2.7e-33$ — same solution within fit precision, not bit-identical; the *historical real-node* measurement (spike-regime backtest nodes, June-2026 harness) was 26.3 → 10.2 ms per fit (≈ 2.6×) — the finding that motivated shipping it. Gated to the var-swap/prior-free configuration; hybrid (analytic core + FD extrapolation rows) under `extrapEnforce`.

3. **Structural validity by construction.** The reparametrization removes the box-constraint handling entirely, so each LM step is a plain linear solve.

4. **Geometric warm start.** Reading (a, m) from the vertex and (b, ρ) from the wing slopes lands θ_0 close enough that liquid smiles converge in a single pass.

C Reference implementation: reverse map and Jacobian core

The raw slice evaluates in one line, $w(k) = a + b(\rho(k-m) + \sqrt{(k-m)^2 + \sigma_{\text{SVI}}^2})$; the reverse conversion (4) reads the five jump-wing handles off it. Composing with listing 1 recovers the original handles to 10^{-10}.

Listing 2: The reverse map raw → SVI-JW — equations (4). The backend ships only the forward converter; this reverse listing is the one the figure generator runs.

```
def w_svi(k, a, b, rho, m, sig):
    # the hyperbola w(k); sig is the SVI width
    km = k - m
    return a + b*(rho*km + np.sqrt(km*km + sig**2))

def raw_to_jw(t, a, b, rho, m, sig):
    # (a, b, rho, m, sig) -> (v, psi, p, c, vtilde)
    w0 = w_svi(0.0, a, b, rho, m, sig); sq = np.sqrt(w0)
    v = w0/t
    psi = b/(2*sq)*(rho - m/np.sqrt(m*m + sig**2))
    p, c = b*(1 - rho)/sq, b*(1 + rho)/sq
    vt = (a + b*sig*np.sqrt(1 - rho*rho))/t
    return v, psi, p, c, vt
```

The production Jacobian core (section 5.3) is equally short — the raw partials (11) times the reparametrization factors (13), then the IV chain rule (12) for the data rows (distilled from `models/svi_jw/jacobian.py`, which it matches to machine precision):

Listing 3: The analytic Jacobian's core: $\mathrm{d}w/\mathrm{d}\theta$ after the chain rule, and the IV data rows.

```
def dw_dtheta(k, a, b, rho, m, sig):
```

```
    # d w / d theta, theta = (a, th_b, th_rho, m, th_sig)
    x = k - m
    r = np.sqrt(x*x + sig*sig)
    sig_b = 1.0 - np.exp(-b)        # d softplus / d th_b
    J = np.empty((k.size, 5))
    J[:, 0] = 1.0                   # d/da
    J[:, 1] = (rho*x + r)*sig_b     # d/db . db/dth_b
    J[:, 2] = (b*x)*(1 - rho*rho)   # d/drho . drho/dth_rho
    J[:, 3] = -b*(rho + x/r)        # d/dm
    J[:, 4] = (b*sig/r)*sig         # d/dsig . dsig/dth_sig
    return J

def data_rows(k, t, w, J, sqrt_weights):
    # residual = sqrt(w_i) (sqrt(w(k)/t) - vol_i): chain rule
    model_vol = np.sqrt(np.maximum(w, 1e-12)/t)
    rows = J / (2.0*t*model_vol)[:, None]
    rows[w <= 1e-12] = 0.0          # the floor flattens the gradient
    return sqrt_weights[:, None] * rows
```

References

[1] J. Gatheral. A parsimonious arbitrage-free implied volatility parameterization. Presentation, Global Derivatives, 2004.

[2] J. Gatheral and A. Jacquier. Arbitrage-free SVI volatility surfaces. *Quantitative Finance*, 14(1):59–71, 2014.

[3] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004.

[4] V. Durrleman. From implied to spot volatilities. *Finance and Stochastics*, 14(2):157–177, 2010.